

UNIDAD 9 · SEMANAS 15–16

SO nativos de IA: agentes y LLM en el sistema operativo

📅 Semanas 15–16 · fechas por confirmar </> Laboratorio TD8 (resuelto + desafío IA)

~ 4 animaciones

LABORATORIO TD8 · RESUELTO (NUEVO)

Un SO agéntico con un micro-LLM propio

Windows y las distribuciones de Linux empiezan a alojar agentes de IA dentro del sistema. Este TD construye esa arquitectura en miniatura con todo lo del curso: un **micro-LLM** entrenado para el curso traduce peticiones en español o inglés a una llamada de herramienta; un **demonio** (`agentd`) la recibe por un socket UNIX, la valida con una **política**, pide **consentimiento** si es destructiva y la ejecuta en un proceso encerrado por **Landlock**, dejando todo en una **bitácora de auditoría**.

</> Compilar y probar

Bash

```
make -C tds                                # also builds td8: tinyllm, agentd, agent,
agent_exec
tds/tests/run.sh td8                        # 24 checks: model, requests, policy,
consent, Landlock, audit
cd tds/bin/td8 && ./tinyllm -v "lista los archivos de docs"
```

Requiere Linux ≥ 5.13 con Landlock activo (`cat /sys/kernel/security/lsm`). El entrenamiento es opcional (los pesos vienen incluidos): `make -C tds/td8 train` (NumPy, ≈ 1 h en CPU).

EJERCICIOS

- | | | | |
|---|---|---|----------------------------------|
| 1 | El micro-LLM: datos y entrenamiento | 4 | La política y el consentimiento |
| 2 | Inferencia en C con <code>mmap</code> y caché KV | 5 | Defensa en profundidad: Landlock |
| 3 | <code>agentd</code> : socket, pool de hilos e identidad | 6 | Auditoría y reflexión |

El micro-LLM: datos y entrenamiento

Enunciado. Lee `train/dataset.py` y `train/train.py`. ¿Qué aprende exactamente el modelo? ¿Por qué los nombres de archivo se generan al azar? ¿Cómo se sabe que el gradiente escrito a mano es correcto?

Solución

Python `tds/td8/train/dataset.py`

```
"""Synthetic data for the TD8 micro-LLM: requests in Spanish or English →
tool call.
```

```
Every example is a text of UTF-8 bytes:
```

```
Q: lista los archivos de docs\n
A: list_files("docs")\n
```

```
The model only learns to translate the intent into ONE call from a closed
catalogue of tools
(the equivalent of the MCP "connectors" of an agentic OS). File and folder
names are generated
at random so that the model learns to COPY them from the request (attention),
not to memorise them.
None of this is safe by itself: the OS decides whether to run the call
(policy + Landlock in agentd).
"""
```

- Un transformer tipo GPT **a nivel de byte** (vocabulario 256, contexto 128, d = 64, 4 cabezas, 3 capas, ≈ 175 mil parámetros), entrenado con NumPy puro (sin PyTorch) y retropropagación escrita a mano; `python3 train.py --check` compara el gradiente analítico con diferencias finitas.
- Aprende a traducir una petición a **una** llamada de un catálogo cerrado de 12 herramientas. Los nombres de archivo al azar obligan al modelo a **copiarlos** de la petición (atención) en lugar de memorizarlos.
- **Resultado real** (`tds/td8/weights/train_log.json`): 174 656 parámetros, 12 000 pasos (≈ 87 min en 4 hilos de CPU), **99,0 %** de coincidencia exacta sobre 500 peticiones nuevas; la herramienta más difícil es `write_note` (89 %), porque hay que copiar dos argumentos. Pesos: 698 688 bytes.

- **Lección de datos:** la versión anterior (v1) acertaba 99,4 % en su prueba, pero en el laboratorio convertía `viejo.txt` en `viefe.txt` y no reconocía `atajo`: había visto pocos nombres así. En lugar de cambiar la red se amplió el generador de datos (nombres con todo el alfabeto, sin extensión). El modelo solo es tan bueno como sus datos, y por eso `agentd` nunca confía en él.
- Los datos incluyen a propósito rutas peligrosas (`/etc/passwd`, `../..`): el modelo debe copiarlas fielmente, porque decidir si son aceptables **no es su trabajo**.

Míralo por dentro en la animación A9.2 →

TD8 · 2

● Resuelto · probado

Inferencia en C con `mmap` y caché KV

Enunciado. Explica cómo `tinylm.c` carga los pesos sin copiarlos y por qué cada token nuevo cuesta una sola pasada por la red. Ejecuta `./tinylm -v` y relaciona sus métricas con las unidades 4 y 6. Comprueba que C y NumPy calculan lo mismo.

Solución

```
C tds/td8/tinylm.c

int tl_load(tl_model *m, const char *path)
{
    memset(m, 0, sizeof *m);
    int fd = open(path, O_RDONLY);
    if (fd == -1)
        return -1;
    struct stat st;
    if (fstat(fd, &st) == -1 || st.st_size < HEADER) {
        close(fd);
        errno = EINVAL;
        return -1;
    }
    /* MAP_PRIVATE + PROT_READ: nobody can modify the weights through this
    mapping. */
    void *map = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd,
0);
    close(fd);
    /* the mapping keeps the
    file alive */
}
```

```

if (map == MAP_FAILED)
    return -1;

const unsigned char *raw = map;
int32_t h[7];
memcpy(h, raw + 8, sizeof h);
if (memcmp(raw, "FSOLLM01", 8) != 0 || h[0] != 1 || h[4] > TL_MAX_LAYERS
|| h[3] % h[5] != 0) {
    munmap(map, (size_t)st.st_size);
    errno = EINVAL;
    return -1;
}
m->vocab = h[1]; m->ctx = h[2]; m->d = h[3]; m->n_layer = h[4]; m->n_head
= h[5]; m->d_ff = h[6];
m->map = map;
m->map_size = (size_t)st.st_size;

/* The pointers point INTO the mapping: no copy (the file is little-
endian float32). */
const float *p = (const float *) (raw + HEADER);
const int d = m->d, f = m->d_ff;
#define TAKE(dst, n) do { (dst) = p; p += (n); } while (0)
TAKE(m->wte, (long)m->vocab * d);
TAKE(m->wpe, (long)m->ctx * d);
for (int l = 0; l < m->n_layer; l++) {
    tl_layer *L = &m->layer[l];
    TAKE(L->ln1_g, d); TAKE(L->ln1_b, d);
    TAKE(L->w_qkv, (long)d * 3 * d); TAKE(L->b_qkv, 3 * d);
    TAKE(L->w_o, (long)d * d); TAKE(L->b_o, d);
    TAKE(L->ln2_g, d); TAKE(L->ln2_b, d);
    TAKE(L->w_fc, (long)d * f); TAKE(L->b_fc, f);
    TAKE(L->w_pr, (long)f * d); TAKE(L->b_pr, d);
}
TAKE(m->lnf_g, d); TAKE(m->lnf_b, d);
#undef TAKE
m->n_params = (long)(p - (const float *) (raw + HEADER));
if ((size_t)HEADER + (size_t)m->n_params * sizeof(float) != m->map_size)
{
    munmap(map, m->map_size);
    errno = EINVAL;
    return -1;
}
return 0;
}

```

Salida real

```
$ ./tinyllm -v "lista los archivos de docs"
step 0: 'l' 100.0% 'f' 0.0% 'r' 0.0%
step 1: 'i' 100.0% 'q' 0.0% '4' 0.0%
step 2: 's' 100.0% 'g' 0.0% 'q' 0.0%
step 3: 't' 100.0% 'k' 0.0% 'b' 0.0%
step 4: '_' 100.0% '-' 0.0% 'k' 0.0%
step 5: 'f' 100.0% '2' 0.0% 'c' 0.0%
step 6: 'i' 100.0% 'o' 0.0% 'n' 0.0%
step 7: 'l' 100.0% 'L' 0.0% 's' 0.0%
step 8: 'e' 100.0% 'o' 0.0% 'f' 0.0%
step 9: 's' 100.0% '(' 0.0% 'q' 0.0%
step 10: '(' 100.0% ' ' 0.0% 'g' 0.0%
step 11: '"' 100.0% ' ' 0.0% 'R' 0.0%
step 12: 'd' 100.0% 'k' 0.0% 'b' 0.0%
step 13: 'o' 100.0% 'i' 0.0% 'e' 0.0%
step 14: 'c' 100.0% 'k' 0.0% 'f' 0.0%
step 15: 's' 100.0% 'q' 0.0% 'f' 0.0%
step 16: '"' 100.0% 'z' 0.0% 'k' 0.0%
step 17: ')' 100.0% ',' 0.0% ':' 0.0%
step 18: '\n' 100.0% 'A' 0.0% ' ' 0.0%
list_files("docs")
```

```
model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

Salida real

```
$ python3 train/compare.py ./tinyllm 200
200/200 respuestas idénticas entre NumPy y C
```

`mmap` con `PROT_READ` y `MAP_PRIVATE` devuelve una dirección; los punteros de cada capa apuntan *dentro* del archivo mapeado. Nada se lee hasta que el primer token toca los pesos: por eso aparecen fallos de página durante la primera inferencia y no durante la carga. La caché KV guarda, por capa y por posición, las claves y los valores ya calculados: cada byte nuevo solo calcula su propia fila. La inferencia en C y la de NumPy dan **las mismas respuestas** en 200 peticiones nuevas.

agentd : socket, pool de hilos e identidad

Enunciado. Lanza `agentd` con un espacio de trabajo y envíale peticiones con `agent`. Identifica en el código qué parte viene de cada unidad del curso. ¿Cómo sabe `agentd` quién le habla?

Solución

C tds/td8/agentd.c

```
static void serve(int client)
{
    struct ucred cred = { 0 };
    socklen_t len = sizeof cred;
    if (getsockopt(client, SOL_SOCKET, SO_PEERCREDS, &cred, &len) == -1)
        return;

    char req[MAX_REQ], out[256], why[PATH_MAX + 64];
    if (read_line(client, req, sizeof req) == 0)
        return;

    /* 1. The model proposes. */
    int steps = tl_generate(&model, req, out, sizeof out, NULL, NULL);
    if (steps < 0) {
        send_str(client, "error: request too long\n");
        audit(cred.uid, cred.pid, req, "-", "deny", "too long");
        return;
    }
    send_fmt(client, "model: %s\n", out);

    /* 2. The operating system disposes. */
    call_t c;
    if (parse_call(out, &c) == -1) {
        send_str(client, "policy: deny (the model produced an invalid\n");
        audit(cred.uid, cred.pid, req, out, "deny", "invalid call");
        return;
    }
    decision_t d = policy(&c, why, sizeof why);
    send_fmt(client, "policy: %s (%s)\n", DECISION[d], why);
    if (d == DENY) {
```

```

        audit(cred.uid, cred.pid, req, out, "deny", why);
        return;
    }
    if (d == ASK) {
        char answer[16];
        send_fmt(client, "ask: allow the agent to run %s? [y/N]\n", out);
        read_line(client, answer, sizeof answer);
        if (answer[0] != 'y' && answer[0] != 'Y') {
            send_str(client, "cancelled by the user\n");
            audit(cred.uid, cred.pid, req, out, "ask", "refused by the
user");
            return;
        }
    }

    /* 3. Sandboxed execution. */
    const char *outcome = run_tool(client, &c);
    audit(cred.uid, cred.pid, req, out, d == ASK ? "ask:approved" : "allow",
outcome);
}

```

Salida real

```

$ ./agent "lista los archivos de docs"
model: list_files("docs")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
2 entries in docs
-      0  plan.md
-      0  syllabus.pdf

```

Salida real

```

$ ./agent "muéstrame notas.txt"
model: read_file("notas.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Laboratorio TD8
fork, exec, pipe, mmap, sockets

```

Salida real

```

$ ./agent "busca 'error' en log.txt"
model: search("error", "log.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
log.txt:2: error: disk full
log.txt:4: error: timeout
2 matching line(s) for "error"
    
```

Salida real

```

$ ./agent "¿qué versión del núcleo tengo?"
model: system_info()
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Linux 7.0.0-34-generic (x86_64), 28 CPUs
memory: 32.8 GB total, 6.9 GB free; 3110 processes; up 4 h 7 min
    
```

Salida real

```

$ for i in $(seq 8); do ./agent "lista mis archivos" & done; wait
8
    
```

PIEZA DE AGENTD

UNIDAD

Socket UNIX con permisos 0600	U8 · U3
<code>SO_PEERCRED</code> : uid y pid del cliente, dados por el núcleo	U8
Pool de hilos con cola acotada (semáforos + mutex)	U7
Pesos compartidos con <code>mmap</code>	U6
<code>fork</code> + <code>execv</code> del ejecutor; salida por una tubería	U4 · U5
<code>SIGTERM</code> para terminar; <code>SIGPIPE</code> ignorada	U6

TD8 · 4

● Resuelto · probado

La política y el consentimiento

Enunciado. Pide al agente que borre un archivo del espacio de trabajo (respondiendo sí y no), que borre `/etc/passwd`, que lea `../../etc/shadow` y que lea `atajo`, un enlace simbólico a `/etc/passwd` dentro del espacio de trabajo. ¿Quién decide en cada caso: el modelo o el SO?

Solución



C tds/td8/agentd.c

```
static int inside_workspace(const char *rel, char *why, size_t whysz)
{
    if (rel[0] == '/' || rel[0] == '~') {
        snprintf(why, whysz, "absolute path outside the workspace");
        return 0;
    }
    if (strcmp(rel, "..") == 0 || strncmp(rel, "../", 3) == 0 || strstr(rel,
"/../") || strstr(rel, "/..") ) {
        snprintf(why, whysz, "'..' leaves the workspace");
        return 0;
    }
    /* Resolve symbolic links: the file (or, if it does not exist yet, its
directory). */
    char full[PATH_MAX * 2], real[PATH_MAX];
    snprintf(full, sizeof full, "%s/%s", workspace, rel);
    if (realpath(full, real) == NULL) {
        char *slash = strrchr(full, '/');
        *slash = '\0';
        if (realpath(full, real) == NULL) {
            snprintf(why, whysz, "directory does not exist");
            return 0;
        }
    }
    size_t n = strlen(workspace);
    if (strncmp(real, workspace, n) != 0 || (real[n] != '\0' && real[n] !=
'/')) {
        snprintf(why, whysz, "resolves to %s, outside the workspace", real);
        return 0;
    }
    return 1;
}

static decision_t policy(const call_t *c, char *why, size_t whysz)
{
    const tool_t *t = c->tool;
    if (strcmp(t->name, "none") == 0) {
        snprintf(why, whysz, "no tool for this request");
        return DENY;
    }
    if (policy_on && t->path_arg >= 0 && !inside_workspace(c->args[t
->path_arg], why, whysz))
        return DENY;
    if (t->consent) {
```

```
    snprintf(why, whysz, "destructive action: needs the user's consent");
    return ASK;
}
    snprintf(why, whysz, policy_on ? "tool in the catalogue, path inside the
workspace" : "POLICY DISABLED");
    return ALLOW;
}
```

Salida real

```
$ ./agent -n "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> n (automatic answer)
cancelled by the user
```

Salida real

```
$ ./agent -y "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> y (automatic answer)
[sandbox: Landlock ABI 8, workspace only]
viejo.txt deleted
```

Salida real

```
$ ./agent "borra /etc/passwd"
model: delete_file("/etc/passwd")
policy: deny (absolute path outside the workspace)
```

Salida real

```
$ ./agent "lee ../../etc/shadow"
model: read_file("../../etc/shadow")
policy: deny ('..' leaves the workspace)
```

Salida real

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: deny (resolves to /etc/passwd, outside the workspace)
```

Salida real

```
$ ./agent "cuéntame un chiste"
model: none()
policy: deny (no tool for this request)
```

El modelo propone siempre lo que le pidieron —incluido `delete_file("/etc/passwd")`—; quien decide es `agentd`: rutas absolutas, `..` y enlaces que escapan (resueltos con `realpath`) se niegan; lo destructivo pregunta al usuario.

Pruébalo con tus propias peticiones en la animación A9.4 →

TD8 · 5

● Resuelto · probado

Defensa en profundidad: Landlock

Enunciado. Reinicia `agentd` con `--no-policy` (simula un error en la política) y repite los ataques. Después ejecuta `agent_exec` directamente con y sin `--no-sandbox`. ¿Qué capa te detiene ahora?

Solución

C tds/td8/agent_exec.c

```
static int sandbox(const char *workspace)
{
    int abi = ll_create(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return 0;

    __u64 fs = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR | LANDLOCK_ACCESS_FS_REMOVE_DIR |
LANDLOCK_ACCESS_FS_REMOVE_FILE |
```

```

        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
LANDLOCK_ACCESS_FS_MAKE_REG |
        LANDLOCK_ACCESS_FS_MAKE_SOCKET | LANDLOCK_ACCESS_FS_MAKE_FIFO |
LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;
    if (abi ≥ 2) fs |= LANDLOCK_ACCESS_FS_REFER;
    if (abi ≥ 3) fs |= LANDLOCK_ACCESS_FS_TRUNCATE;
    if (abi ≥ 5) fs |= LANDLOCK_ACCESS_FS_IOCTL_DEV;

    struct landlock_ruleset_attr attr = { .handled_access_fs = fs };
    size_t size = sizeof(__u64); /* ABI 1-3:
only the fs field */
    if (abi ≥ 4) {
        attr.handled_access_net = LANDLOCK_ACCESS_NET_BIND_TCP |
LANDLOCK_ACCESS_NET_CONNECT_TCP;
        size = offsetof(struct landlock_ruleset_attr, handled_access_net) +
sizeof(__u64);
    }
    if (abi ≥ 6) {
        attr.scoped = LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET |
LANDLOCK_SCOPE_SIGNAL;
        size = sizeof attr;
    }
    int rs = ll_create(&attr, size, 0);
    exit_if(rs == -1, "landlock_create_ruleset");

    /* Everything inside the workspace (minus executing programs), read-only
/proc. */
    exit_if(ll_add_path(rs, workspace, fs & ~
(__u64)LANDLOCK_ACCESS_FS_EXECUTE) == -1, "landlock_add_rule workspace");
    exit_if(ll_add_path(rs, "/proc", LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR) == -1,
        "landlock_add_rule /proc");

    exit_if(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1, "prctl");
    exit_if(syscall(SYS_landlock_restrict_self, rs, 0) == -1,
"landlock_restrict_self");
    close(rs);
    return abi;
}

static void limits(void)
{
    struct rlimit cpu = { 2, 3 }, fsize = { 1 << 20, 1 << 20 }, nofile = {
64, 64 };

```

```
exit_if(setrlimit(RLIMIT_CPU, &cpu) = -1, "setrlimit cpu"); /*
SIGXCPU after 2 s */
exit_if(setrlimit(RLIMIT_FSIZE, &fsize) = -1, "setrlimit fsize"); /*
SIGXFSZ past 1 MiB */
exit_if(setrlimit(RLIMIT_NOFILE, &nofile) = -1, "setrlimit nofile");
alarm(5); /*
SIGALRM: real time */
}
```

Salida real

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: atajo: Permission denied
```

Salida real

```
$ ./agent "lee /etc/passwd"
model: read_file("/etc/passwd")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/passwd: Permission denied
```

Salida real

```
$ ./agent_exec workspace read_file /etc/hostname; ./agent_exec --no-sandbox
workspace read_file /etc/hostname
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/hostname: Permission denied
[sandbox: DISABLED]
host
```

Antes de tocar nada, `agent_exec` se encierra: límites de CPU, tamaño de archivos y descriptores (`setrlimit`), una alarma de 5 s, `no_new_privs` y una regla de Landlock que solo permite el espacio de trabajo (y `/proc` en lectura), niega conexiones TCP y el envío de señales fuera del sandbox. Aunque la política esté desactivada, el **núcleo** responde `Permission denied`. El mismo programa, sin sandbox, sí lee el archivo.

TD8 · 6

● Resuelto · probado

Auditoría y reflexión

Enunciado. Lee `audit.log`. ¿Qué información permite reconstruir lo que hizo el agente y en nombre de quién? ¿Qué le falta a este SO agéntico para usarse de verdad?

Solución



Salida real

```

2026-09-30T13:14:12 uid=1000 pid=364726 request="lista los archivos de docs"
call=list_files("docs") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364735 request="muéstrame notas.txt"
call=read_file("notas.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364744 request="¿cuántas líneas tiene
main.c?" call=count_lines("main.c") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364753 request="busca 'error' en log.txt"
call=search("error", "log.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364762 request="anota 'examen el lunes' en
agenda.md" call=write_note("agenda.md", "examen el lunes") decision=allow
outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364771 request="crea la carpeta proyectos"
call=make_dir("proyectos") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364780 request="¿cuánto espacio libre
queda?" call=disk_usage() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364789 request="¿qué versión del núcleo
tengo?" call=system_info() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364798 request="show running processes"
call=processes() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364807 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask outcome=refused by the user
2026-09-30T13:14:13 uid=1000 pid=364815 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask:approved outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364824 request="borra /etc/passwd"
call=delete_file("/etc/passwd") decision=deny outcome=absolute path outside
the workspace
2026-09-30T13:14:13 uid=1000 pid=364832 request="lee ../etc/shadow"
call=read_file("../etc/shadow") decision=deny outcome='..' leaves the
workspace
2026-09-30T13:14:13 uid=1000 pid=364840 request="lee atajo"
call=read_file("atajo") decision=deny outcome=resolves to /etc/passwd,
outside the workspace
2026-09-30T13:14:13 uid=1000 pid=364848 request="cuéntame un chiste"
call=none() decision=deny outcome=no tool for this request
2026-09-30T13:14:13 uid=1000 pid=364873 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364875 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364877 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364876 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364880 request="lista mis archivos"

```

```

call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364879 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364881 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364878 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok

```

- Cada línea tiene fecha, uid y pid del cliente (dados por el núcleo, no por el cliente), la petición, la llamada propuesta, la decisión y el resultado: permite distinguir lo que hizo el agente de lo que pidió el usuario, como exige Windows con sus cuentas de agente.
- Le falta, entre otras cosas: un usuario propio para el agente (hoy corre con el del docente), un filtro `seccomp`, límites de memoria con `cgroups`, protección de la bitácora contra el propio agente y defensa frente a la inyección de instrucciones cuando el agente lea el contenido de archivos.

Lista de cotejo

Desafío con IA generativa · Una herramienta nueva para el agente... y un ataque ^

Objetivo. Agregar al SO agéntico una herramienta `copy_file(origen, destino)`: generar datos, reentrenar el micro-LLM, extender la política y el ejecutor. Después intentar que el agente copie un archivo fuera del espacio de trabajo y comprobar qué capa lo detiene.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Estoy trabajando con un SO agéntico de juguete en C: un micro-LLM (`tinyllm.c`) traduce peticiones a llamadas de herramientas, `agentd` aplica una política y `agent_exec` ejecuta la herramienta dentro de un sandbox Landlock [pega `dataset.py`, la tabla `TTOOLS` de `agentd.c` y `main()` de `agent_exec.c`]. Quiero agregar `copy_file(origen, destino)`. Guíame para: (1) añadir plantillas en español e inglés a `dataset.py`; (2) reentrenar y medir la exactitud antes y después; (3) extender la política para que AMBOS argumentos estén dentro del espacio de trabajo; (4) implementar la copia en `agent_exec`. Luego ayúdame a diseñar 5 peticiones maliciosas (rutas absolutas, '..', enlaces simbólicos, inyección en el texto) y predice cuál capa detiene cada una: el parser, la política o Landlock.

Cómo verificar el resultado

- La exactitud del modelo reentrenado sobre las herramientas anteriores no baja más de 2 puntos (`train.py --eval`).
- `copy_file` con un destino fuera del espacio de trabajo termina en `policy: deny`, y con `--no-policy` en `Permission denied` (Landlock).
- La bitácora `audit.log` registra los 5 intentos con su decisión.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.